计算机视觉基础：Canny 边缘检测与 Harris 角点检测

特征提取总结报告

2026 年 3 月 19 日

目录

1 总述 **3**

2 Canny 边缘检测 **3**

2.1 三大核心设计准则 3

2.2 完整算法步骤（逐环节详解） 3

2.2.1 步骤 1：高斯平滑**去噪** 3

2.2.2 步骤 2：计算**梯度**幅值与方向 4

2.2.3 步骤 3：**非极大值抑制（NMS）** 4

2.2.4 步骤 4：**双阈值**筛选与边缘连接 5

2.3 Canny 算法的特性与优缺点 6

3 Harris 角点检测 **6**

3.1 核心数学原理 6

3.1.1 步骤 1：灰度变化的量化公式 7

3.1.2 步骤 2：泰勒展开与二阶矩矩阵（结构张量） 7

3.1.3 步骤 3：基于特征值的像素分类 8

3.1.4 步骤 4：角点响应函数 R 8

3.1.5 步骤 5：角点筛选与非极大值抑制 9

3.2 Harris 角点的不变性与等变性 9

3.2.1 （1）对图像平移：等变 9

3.2.2 （2）对图像旋转：等变 9

3.2.3 （3）对仿射亮度变化（$I \to aI + b$）：部分不变 9

3.2.4 （4）对图像尺度变化：既不不变，也不等变 9

3.3 Harris 算法的特性与优缺点 10

4 Canny 边缘检测 vs Harris 角点检测核心对比 **10**

5 实验一：Harris 角点检测 (Harris Corner Detection) 10
5.1 实现逻辑说明 10
5.2 Python 代码实现 11

6 Harris 角点检测参数影响深度分析 12
6.1 参数 k (响应常数) 的影响 12
6.2 窗口大小 (Window Size) 的影响 12
6.3 阈值比例 (Threshold Ratio) 的影响 12
6.4 参数研究总结对比表 13

7 实验二：Canny 边缘检测 (Canny Edge Detection) 13
7.1 算法步骤 13
7.2 Python 代码实现 13

8 差异分析：手动实现 vs OpenCV 内置函数 14

1 总述

Canny 边缘检测与 Harris 角点检测是计算机视觉领域最经典的低层次特征提取算法，二者定位完全不同：**Canny 用于检测图像中亮度突变的边缘轮廓**，是图像分割、目标识别的基础；**Harris 用于检测图像中二维梯度均剧烈变化的角点（兴趣点）**，是局部特征匹配、图像配准、三维重建的核心基础。

2 Canny 边缘检测

Canny 算法由 John F. Canny 于 1986 年提出，其设计核心是严格遵循三大最优准则，至今仍是工业界和学术界最通用、效果最稳定的边缘检测算法。

2.1 三大核心设计准则

算法的全流程均围绕这三大准则设计，也是其效果远超基础边缘检测算子的核心原因：

1. **低错误率准则**：精准检测出真实边缘，无漏检、无误检，最大程度降低噪声带来的假边缘。
2. **高定位精度准则**：检测出的边缘点位置，与真实边缘的物理中心位置偏差最小。
3. **单边缘响应准则**：对同一条真实边缘，仅生成一个像素宽度的响应，避免出现多边缘重复检测。

2.2 完整算法步骤（逐环节详解）

Canny 算法是一套流水线式的处理流程，每一步都对应上述三大准则，缺一不可。

2.2.1 步骤 1：高斯平滑去噪

边缘检测的核心是计算图像灰度的梯度，而梯度对随机噪声极其敏感，噪声会导致虚假梯度峰值，因此必须先对图像进行高斯平滑滤波，抑制噪声。

- 核心操作：用二维高斯核与原图像做卷积，实现加权平滑（中心像素权重更高，边缘权重更低）。
- 二维高斯核公式：
$$G(x,y,\sigma)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{1}$$
其中，(x,y) 是像素坐标，σ 是高斯核的标准差，决定平滑程度。
- 关键说明：
  - σ 越大，高斯核的平滑范围越大，去噪能力越强，但会模糊边缘细节；σ 越小，细节保留越好，但抗噪能力下降。
  - 工程中常用 3x3、5x5 的高斯核，核尺寸通常取 $6\sigma+1$ 左右（保证高斯权重覆盖有效范围）。

2.2.2 步骤 2：计算梯度幅值与方向

平滑后的图像，计算每个像素在 x（水平）、y（垂直）方向的灰度梯度，得到梯度的幅值和方向，这一步是定位边缘的基础。

- 梯度计算：工程中最常用 **Sobel 算子**计算梯度，Sobel 算子兼具梯度计算和轻微平滑的能力，抗噪性优于 Prewitt、Roberts 算子。
  - x 方向 Sobel 核（检测垂直边缘）：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

  - y 方向 Sobel 核（检测水平边缘）：

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- 梯度幅值（边缘强度）：

$$M(x,y) = \sqrt{G_x(x,y)^2 + G_y(x,y)^2} \tag{2}$$

  幅值越大，代表该像素处的灰度突变越剧烈，是边缘的概率越高。

- 梯度方向：

$$\theta(x,y) = \arctan 2(G_y(x,y), G_x(x,y)) \tag{3}$$

  梯度方向**始终垂直于边缘的走向**，是后续非极大值抑制的核心依据。例如：水平边缘的梯度方向为垂直方向（90°/270°），垂直边缘的梯度方向为水平方向（0°/180°）。

2.2.3 步骤 3：非极大值抑制（NMS）

这一步是实现**单边缘响应准则**的核心，目标是将粗的、多像素宽的边缘，细化为单像素宽度的精准边缘。

- 核心原理：遍历图像中的每个像素，判断该像素的梯度幅值，是否是其**梯度方向上相邻两个像素**的局部最大值。若不是最大值，则抑制该像素（灰度置 0，判定为非边缘）；若是最大值，则保留。
- 工程实现细节：
  1. 将梯度方向 θ 量化为 4 个主方向（0°、45°、90°、135°），覆盖所有边缘走向；
  2. 对每个像素，根据量化后的梯度方向，找到对应的邻域对比像素：
     - 0°（水平梯度）：对比当前像素的左右邻域像素；

- 45°：对比当前像素的右上、左下邻域像素；
- 90°（垂直梯度）：对比当前像素的上下邻域像素；
- 135°：对比当前像素的左上、右下邻域像素；

3. 仅保留梯度幅值的局部极大值，其余像素全部置 0，最终得到单像素宽的候选边缘。

在非极大值抑制阶段，算法根据计算出的梯度方向 θ 将其归类为四个离散方向，并据此选择比较的邻域像素。下表详细列出了方向区间、邻域位置及物理意义：

backcolour **量化方向**	**梯度方向 θ 对应区间**	**比较邻域位置**	**梯度与边缘关系说明**
0°	$[0°, 22.5°] \cup [157.5°, 180°]$	**左、右**	**梯度水平**，说明边缘走向是垂直的。
45°	$[22.5°, 67.5°]$	**右上、左下**	**梯度斜向上**，说明边缘走向是斜向下的。
90°	$[67.5°, 112.5°]$	**上、下**	**梯度垂直**，说明边缘走向是水平的。
135°	$[112.5°, 157.5°]$	**左上、右下**	**梯度斜向下**，说明边缘走向是斜向上的。

表 1: Canny NMS 方向量化及邻域像素比对逻辑表

2.2.4 步骤 4：双阈值筛选与边缘连接

这一步是实现**低错误率准则**的核心，解决了单阈值检测中“阈值过高漏检边缘、阈值过低引入噪声”的矛盾。

- 核心操作：设置高低两个阈值 T_h（高阈值）和 T_l（低阈值），工程中通常取 $T_h = 2 \sim 3 \times T_l$，按以下规则筛选边缘：
  1. **强边缘**：像素梯度幅值 $M > T_h$，判定为真实边缘，直接保留；
  2. **无效像素**：像素梯度幅值 $M < T_l$，判定为非边缘/噪声，直接抑制（置 0）；
  3. **弱边缘**：像素梯度幅值 $T_l < M < T_h$，仅当该像素的 8 邻域内存在强边缘像素时，才判定为真实边缘并保留；否则判定为虚假边缘，抑制置 0。
- 关键说明：高阈值保证了检测到的边缘的可靠性，低阈值保证了真实边缘的连续性，通过邻域连通性判断，将断裂的弱边缘与强边缘连接，同时剔除孤立的噪声弱边缘。

2.3 Canny 算法的特性与优缺点

- **核心优势**：
  1. 严格遵循三大最优准则，检测效果远超 Sobel、Laplacian 等基础边缘检测算法；
  2. 边缘定位精准，输出单像素宽的连续边缘，无冗余响应；
  3. 对噪声鲁棒性强，高斯平滑 + 双阈值筛选双重抗噪；
  4. 可解释性强，参数可控，适配绝大多数自然图像场景。
- **局限性**：
  1. 超参数依赖强：高斯核 σ、高低阈值需要根据图像场景手动调整，无通用最优参数；
  2. 计算量大于基础边缘检测算子，实时性要求极高的场景需做优化；
  3. 对纹理密集、边缘模糊的图像，容易出现过度抑制或边缘断裂；
  4. 不具备尺度不变性，图像缩放后边缘检测效果会下降。

3 Harris 角点检测

Harris 角点检测由 Chris Harris 和 Mike Stephens 于 1988 年提出，是对 Moravec 角点检测的重大改进，核心思想是**角点是在任意方向移动局部窗口，窗口内灰度都会发生剧烈变化的像素点**。

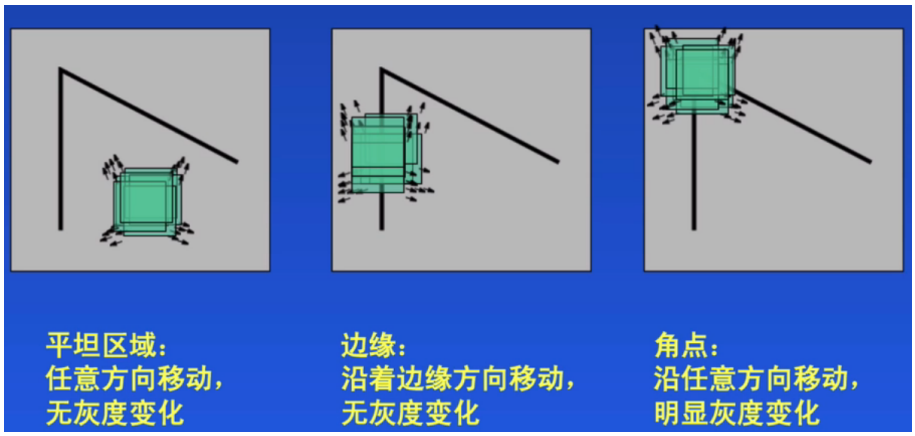

图 1: 原理

3.1 核心数学原理

角点的本质是：二维平面中，两个正交方向的灰度梯度都极大的点。我们通过量化窗口平移后的灰度变化，来判断该点是否为角点。

3.1.1 步骤 1：灰度变化的量化公式

定义窗口 $w(x,y)$（通常为高斯窗口，中心像素权重更高）在平移 (u,v) 后，窗口内的灰度变化值 $E(u,v)$：

$$E(u,v) = \sum_{x,y} w(x,y) \cdot [I(x+u, y+v) - I(x,y)]^2 \tag{4}$$

其中，$I(x,y)$ 是原图像灰度值，$w(x,y)$ 是窗口权重函数，$E(u,v)$ 越大，代表窗口平移后的灰度变化越剧烈。

3.1.2 步骤 2：泰勒展开与二阶矩矩阵（结构张量）

由于窗口平移量 (u,v) 极小，我们对 $I(x+u, y+v)$ 做一阶泰勒展开：

$$I(x+u, y+v) \approx I(x,y) + u \cdot I_x + v \cdot I_y \tag{5}$$

其中，$I_x = \frac{\partial I}{\partial x}$ 是 x 方向灰度梯度，$I_y = \frac{\partial I}{\partial y}$ 是 y 方向灰度梯度。

$$\begin{aligned}
E(u,v) &= \sum_{x,y} w(x,y)[I(x+u,y+v) - I(x,y)]^2 \\
&= \sum_{x,y} w(x,y)[I(x,y) + I_x u + I_y v - I(x,y)]^2 \\
&= \sum_{x,y} w(x,y)[I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2] \\
&= \sum_{x,y} w(x,y)\,[u,v] \begin{bmatrix} I_x^2, I_x I_y \\ I_x I_y, I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} \\
&= [u,v] \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2, I_x I_y \\ I_x I_y, I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}
\end{aligned}$$

图 2: 推导过程

将泰勒展开式代入 $E(u,v)$，化简后得到核心公式：

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} \tag{6}$$

其中，**二阶矩矩阵 M（结构张量）**是 Harris 算法的核心，定义为：

$$M = \sum_{x,y} w(x,y) \cdot \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \tag{7}$$

M 是 2×2 的实对称矩阵，它完整描述了窗口内像素的梯度分布，其特征值直接决定了该点的类型。

3.1.3 步骤 3：基于特征值的像素分类

对 M 做特征值分解，得到两个正交主方向的特征值 λ_1 和 λ_2，两个特征值分别对应了窗口内灰度变化最大和最小的两个方向的变化强度，由此可以将像素分为三类：

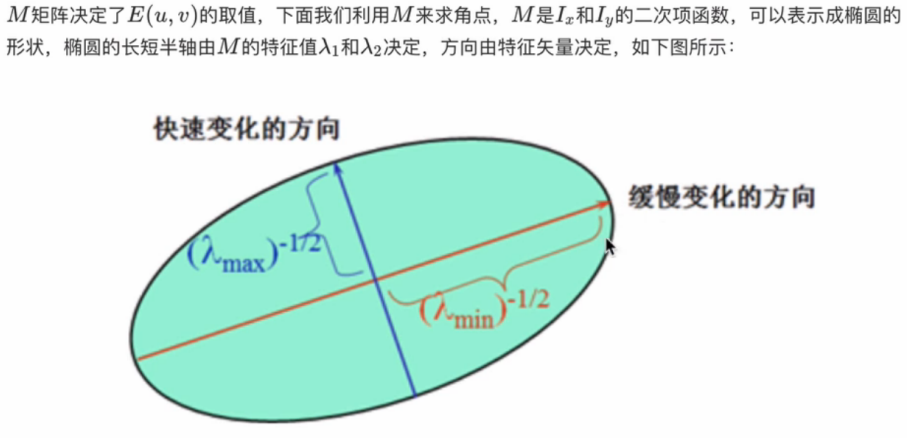

图 3: λ_1 和 λ_2 说明

1. **平坦区域**：λ_1 和 λ_2 都很小，且数值接近。代表窗口内灰度均匀，任意方向平移，$E(u,v)$ 都几乎无变化。
2. **边缘区域**：一个特征值极大，另一个特征值极小。代表窗口仅在一个方向平移时灰度变化剧烈，另一个方向无变化，对应边缘。
3. **角点区域**：λ_1 和 λ_2 都很大，且数值接近。代表窗口在任意方向平移，$E(u,v)$ 都会发生剧烈变化，对应角点。

3.1.4 步骤 4：角点响应函数 R

为了避免计算特征值带来的额外计算量，Harris 设计了无需特征分解的角点响应函数，直接通过 M 的行列式和迹计算响应值：

$$R = \det(M) - k \cdot (\mathrm{trace}(M))^2 \tag{8}$$

其中：

- 矩阵行列式：$\det(M) = \lambda_1 \lambda_2$
- 矩阵的迹：$\mathrm{trace}(M) = \lambda_1 + \lambda_2$
- k 是经验常数，工程中通常取 $0.04 \sim 0.06$，k 越大，对角点的筛选越严格，检测到的角点越少、越尖锐。

3.1.5 步骤 5：角点筛选与非极大值抑制

1. 阈值筛选：设置响应阈值 T，仅当 $R > T$ 时，该像素才被判定为候选角点；$R < 0$ 通常为边缘，$|R|$ 极小为平坦区域。
2. 非极大值抑制：在 3x3/5x5 的邻域内，仅保留响应值 R 的局部极大值，剔除相邻的冗余角点，避免角点聚集，保证角点的定位精度和稀疏性。

3.2 Harris 角点的不变性与等变性

不变性和等变性是特征检测算法的核心性能指标，直接决定了算法对图像变换的鲁棒性。

- 基础定义回顾：
  - **不变性**：图像发生变换后，特征的检测结果（是否为角点、响应值）不发生变化。
  - **等变性（协变性）**：图像发生变换后，特征会出现在变换后的对应位置，不会消失或错位。

  日常表述中，常将二者统称为“不变性”。

3.2.1 （1）对图像平移：等变

图像平移后，梯度计算和窗口函数均具有平移等变性，二阶矩矩阵 M 的数值不变，角点响应值 R 不变，角点会精准出现在平移后的对应位置，不会丢失。

3.2.2 （2）对图像旋转：等变

图像旋转后，二阶矩矩阵 M 的特征向量会同步旋转，但特征值 λ_1 和 λ_2 的数值完全不变，因此角点响应值 R 不变，角点会出现在旋转后的对应位置。对应“二阶矩椭圆旋转但它的形状（即特征值）保持不变”。

3.2.3 （3）对仿射亮度变化（$I \to aI + b$）：部分不变

- 亮度偏移（$I \to I + b$）：完全不变。因为 Harris 仅使用灰度梯度 I_x I_y，常数偏移的导数为 0，M 矩阵和响应值 R 完全不受影响。
- 亮度缩放（$I \to aI$）：部分不变。梯度会同步缩放 a 倍，M 矩阵缩放 a^2 倍，$\det(M)$ 缩放 a^4 倍，$\mathrm{trace}(M)$ 缩放 a^2 倍，最终 R 缩放 a^4 倍。但角点的相对响应排序不变，只要阈值同步调整，角点的位置完全不变。

3.2.4 （4）对图像尺度变化：既不不变，也不等变

这是 Harris 算法最核心的局限性。

- 举例说明：一个小窗口内的角点，当图像放大后，原来的小窗口内仅能覆盖边缘的一部分，原本的角点会被判定为边缘；反之，图像缩小后，原本的边缘会被判定为角点。

- 本质原因：Harris 的窗口尺寸是固定的，无法适配图像缩放后的特征尺度，因此不具备尺度不变性，这也是后续 SIFT、SURF 等尺度不变特征算法的核心改进方向。

3.3 Harris 算法的特性与优缺点

- **核心优势**：
  1. 计算简单高效，仅基于灰度梯度计算，易于工程实现；
  2. 角点定位精度极高，检测到的角点均匀、可重复性强；
  3. 对图像旋转、仿射亮度变化鲁棒性强，适配不同光照、视角旋转的场景；
  4. 对边缘的区分度高，能有效避免将边缘误检为角点。
- **局限性**：
  1. 不具备尺度不变性，图像缩放后检测效果严重下降；
  2. 对噪声敏感，基于梯度的计算易受噪声干扰，通常需要先做高斯平滑预处理；
  3. 响应函数中的 k 值为经验参数，不同场景需要手动调整，无通用最优值；
  4. 单像素响应，对图像的微小形变鲁棒性不足，后续需要配合特征描述子实现匹配。

4 Canny 边缘检测 vs Harris 角点检测核心对比

为了更清晰地展示两种算法的差异，我们通过以下表格进行对比：

特性维度	**Canny 边缘检测**	**Harris 角点检测**
核心目标	检测图像中灰度突变的边缘轮廓	检测图像中二维梯度均剧烈的角点
核心依据	梯度的幅值与方向，单边缘响应	二阶矩矩阵的特征值，多方向灰度变化
输出结果	单像素宽的边缘二值图像	稀疏的角点坐标集合
旋转鲁棒性	等变	等变
亮度鲁棒性	较强	部分不变
尺度不变性	不具备	不具备
核心应用场景	图像分割、轮廓提取、目标检测	图像配准、特征匹配、三维重建、SLAM

表 2: Canny 与 Harris 算法核心特性对比

5 实验一：Harris 角点检测 (Harris Corner Detection)

5.1 实现逻辑说明

本次实验完全按照 Harris 算法的标准步骤手动实现，不调用内置的检测函数。核心逻辑如下：

1. **梯度计算**：利用 Sobel 算子计算图像在 x 和 y 方向的一阶导数 I_x, I_y。
2. **结构张量构造**：计算 I_x^2, I_y^2, I_xI_y 并在指定窗口 (win) 下进行高斯平滑，模拟 M 矩阵的求和过程。
3. **响应值计算**：根据公式 $R = \det(M) - k \cdot (\mathrm{tr}(M))^2$ 计算每个像素的响应。
4. **非极大值抑制 (NMS)**：通过 dilate 操作寻找局部最大值点。
5. **阈值筛选**：利用 threshold_ratio 与全局最大响应值的乘积作为动态阈值。

5.2 Python 代码实现

```python
import cv2
import numpy as np
import os

def harris_core(img, k, win, threshold_ratio):
    """核心算法逻辑：返回标记好角点的图片"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
    Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    Ixx = cv2.GaussianBlur(Ix ** 2, (win, win), 0)
    Iyy = cv2.GaussianBlur(Iy ** 2, (win, win), 0)
    Ixy = cv2.GaussianBlur(Ix * Iy, (win, win), 0)

    det_M = (Ixx * Iyy) - (Ixy ** 2)
    trace_M = Ixx + Iyy
    R = det_M - k * (trace_M ** 2)

    R_max = cv2.dilate(R, None)
    threshold = threshold_ratio * R.max()
    corner_mask = (R == R_max) & (R > threshold)

    result = np.copy(img)
    coords = np.argwhere(corner_mask)
    for y, x in coords:
        # 加大标记半径至 4，确保可视化清晰
        cv2.circle(result, (x, y), 4, (0, 0, 255), -1)
    return result, len(coords)

def run_parameter_experiments(image_path):
    img = cv2.imread(image_path)
```

```
    output_dir = "harris_experiments"
    if not os.path.exists(output_dir): os.makedirs(output_dir)

    # 实验部分：测试 k, Window Size, Threshold Ratio
    k_list = [0.02, 0.04, 0.08, 0.15]
    win_list = [3, 5, 9, 15]
    ratio_list = [0.0001, 0.0005, 0.005, 0.05]
    # ... 循环调用 harris_core 并保存（代码略）...
```

Listing 1: Harris 角点检测核心算法与实验脚本

6 Harris 角点检测参数影响深度分析

在 Harris 算法中，响应函数 $R = \det(M) - k \cdot (\mathrm{trace}(M))^2$ 的表现受多个超参数共同驱动。通过对 k、窗口大小（Window Size）及阈值比例（Threshold Ratio）的对比实验，得出以下分析结论：

6.1 参数 k（响应常数）的影响

k 值主要调节算法对**边缘抑制**的强度。

- **较小的 k（如 0.02）**：算法极其敏感，容易将图像中的直线边缘（仅单方向梯度大的区域）误判为角点，导致结果中出现“边缘堆积”现象。
- **较大的 k（如 0.15）**：惩罚项变大，算法变得极其“挑剔”。它能完美剔除边缘干扰，但会**漏掉对比度较低的真实角点**。

6.2 窗口大小（Window Size）的影响

窗口大小决定了梯度信息的**整合尺度**（Integration Scale）。

- **小窗口（如 3x3）**：定位精度极高，能捕捉微小细节。但对**孤立噪点敏感**，容易产生伪角点。
- **大窗口（如 15x15）**：通过平滑效应增强了抗噪性，提取的是图像的宏观几何特征。然而，这会导致角点位置发生**视觉漂移**（不够精准），且邻近角点容易融合。

6.3 阈值比例（Threshold Ratio）的影响

阈值是筛选特征强度的最后一道关卡，直接决定了角点的**稀疏程度**。

- **低阈值**：检测到的角点数量庞大，涵盖了图像中所有的细微转角，适用于需要详尽特征的匹配任务。
- **高阈值**：仅保留图像中最具代表性的、响应最强的特征点。这极大地**降低了后续算法（如特征匹配）的计算负担**，提高了实时性。

6.4 参数研究总结对比表

下表总结了增加各参数数值时，对检测结果各项指标的影响趋向：

backcolour **调整方向（参数增大）**	**角点数量**	**抗噪能力**	**边缘抑制**	**定位精度**
参数 $k \uparrow$	显著减少	略微增强	**显著变好**	保持稳定
窗口 **Window** $\uparrow$	逐渐减少	**显著增强**	保持稳定	**下降（模糊）**
阈值 **Threshold** $\uparrow$	**大幅减少**	增强	不变	保持稳定

表 3: Harris 角点检测参数影响实验对比总结

7 实验二：Canny 边缘检测 (Canny Edge Detection)

7.1 算法步骤

根据题目要求，手动实现必须包含以下阶段：1. **高斯滤波**：平滑图像以降低噪声。2. **计算梯度幅度与方向**：使用 Sobel 算子。3. **非极大值抑制 (NMS)**：在梯度方向上搜索局部最大值，细化边缘。4. **双阈值检测**：根据高低阈值将像素分类为强边缘、弱边缘或非边缘。5. **滞后边界跟踪**：保留与强边缘相连的弱边缘像素。

7.2 Python 代码实现

```python
def canny_manual(img, low_t, high_t, sigma=1):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), sigma)
    # 梯度计算
    gx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0)
    gy = cv2.Sobel(blurred, cv2.CV_64F, 0, 1)
    mag = np.sqrt(gx**2 + gy**2)
    theta = np.arctan2(gy, gx) * 180 / np.pi
    theta[theta < 0] += 180

    # NMS (简化为4个方向)
    nms = np.zeros_like(mag)
    for i in range(1, mag.shape[0]-1):
        for j in range(1, mag.shape[1]-1):
            angle = theta[i,j]
            q, r = 255, 255
            if (0 <= angle < 22.5) or (157.5 <= angle <= 180):
                q, r = mag[i, j+1], mag[i, j-1]
            elif (22.5 <= angle < 67.5):
```

```
                q, r = mag[i+1, j-1], mag[i-1, j+1]
            elif (67.5 <= angle < 112.5):
                q, r = mag[i+1, j], mag[i-1, j]
            else:
                q, r = mag[i-1, j-1], mag[i+1, j+1]
            if mag[i,j] >= q and mag[i,j] >= r: nms[i,j] = mag[i,j]

    # 双阈值与跟踪
    res = np.zeros_like(nms, dtype=np.uint8)
    res[nms >= high_t] = 255
    res[(nms < high_t) & (nms >= low_t)] = 50
    for i in range(1, res.shape[0]-1):
        for j in range(1, res.shape[1]-1):
            if res[i,j] == 50 and (res[i-1:i+2, j-1:j+2] == 255).any():
                res[i,j] = 255
            elif res[i,j] == 50: res[i,j] = 0
    return res
```

Listing 2: Canny 手动实现与参数搜索

8 差异分析：手动实现 vs OpenCV 内置函数

在实验过程中观察到，即便参数设置完全一致，手动实现的 Canny 与 `cv2.Canny` 的结果仍有细微差异。原因如下：

1. **梯度模长计算 (L1 vs L2 Norm)**: OpenCV 默认使用 $L1$ 范数 ($|G_x| + |G_y|$) 以提升速度。手动实现通常采用数学更精确的 $L2$ 范数 ($\sqrt{G_x^2 + G_y^2}$)。

2. **非极大值抑制的精度**: 手动代码通常将梯度方向简化为四个扇区（0°，45°，90°，135°）。OpenCV 的实现则采用了更精细的亚像素插值，能更准确地定位边缘中心，因此其边缘往往比手动版更细。

3. **数值计算深度**: OpenCV 在 C++ 底层使用了 16 位定点数优化，而 Python 代码依赖 64 位浮点数。这种精度差异会在滞后阈值判定时累积。

4. **边界处理机制**: OpenCV 在进行卷积和平滑时，默认采用镜像填充 (`BORDER_REFLECT`)，而简易的手动实现可能仅做了简单的补零或忽略边缘。